Low Level Design Handbook

PART 1

1. Introduction to Low Level Design

★ What is a System ?

A system is a collection of components / classes that work together to achieve a specific goal.

Example :

E-commerce system, ATM system, Food delivery system, etc.

★ Why do companies ask LLD ?

- ✓ To check your problem solving and design thinking.
- ✓ To verify how well you can break down a problem.
- ✓ To check knowledge of OOPs principles and design patterns.
- ✓ To ensure you can write clean, maintainable and scalable code.

★ Where is LLD Used ?

- Product based companies
- Service based applications
- Real world systems
- Day to day software
- Almost everywhere ☺

★ What is Low Level Design (LLD) ?

LLD is the detailed design of individual modules or components of a system. It describes how each part of the system will be implemented.

Focus :

Classes, objects, methods, data structures, relationships and algorithms.

Characteristics of Good Design

1. Reusability Classes and components should be reusable in different parts of the system.	2. Maintainability Design should be easy to understand, modify and debug.	3. Extensibility System should be easy to extend with new features.
4. Testability Design should be easy to test. Each part should be testable independently.	5. Scalability System should handle increased load and growth.	6. Flexibility Able to adapt to changes in requirements with minimal impact.

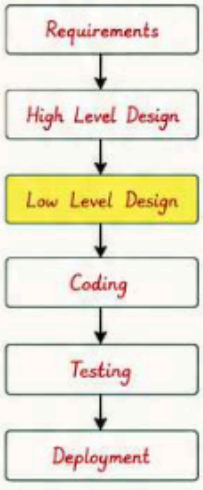
★ LLD vs HLD

Aspect	High Level Design (HLD)	Low Level Design (LLD)
Purpose	Overall system design	Detailed design of modules
Focus	System architecture, components, data flow	Classes, methods, logic, relationships
Abstraction	More abstract	More detailed
Level	Macro level	Micro level
Output	Architecture diagram, modules	Class diagram, sequence diagram, code structure
Done by	Architect / Senior Developer	Developer
Example	Zomato high level architecture	Restaurant, Order, Payment module design

★ LLD in Real World

- Breaking a big problem into smaller pieces.
- Designing classes and their interactions.
- Choosing right data structures and algorithms.
- Following OOP principles and design patterns.
- Making code clean, modular and easy to maintain.

★ LLD Flow



Note :

- ✓ Good design leads to good code.
- ✓ Always think before you code.
- ✓ Focus on simplicity and clarity.
- ✓ Follow SOLID principles.
- ✓ Use design patterns where required.
- ✓ Think from the perspective of maintainability.
- ✓ A well designed system is easy to build, test and scale.

In this Handbook we will cover

- → UML Diagrams
- → SOLID Principles
- → Design Patterns
- → Real world LLD examples
- → Best practices & Tips

GOOD DESIGN IS NOT JUST HOW IT LOOKS,
IT'S HOW WELL IT WORKS AND GROWS. ☺

Low Level Design Handbook

PART 2

2. UML Diagrams (Basics)

★ What is UML ?

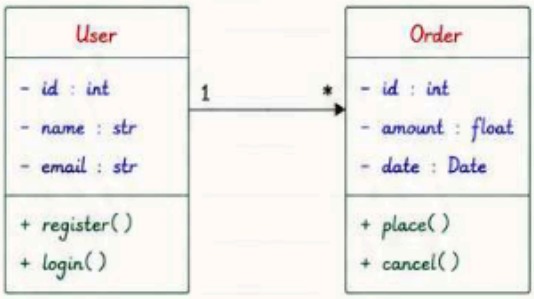
UML (Unified Modeling Language) is a standard way to visualize the design of a system using diagrams.

Why UML ?

- ✓ Helps in visualizing the system.
- ✓ Improves communication between team members.
- ✓ Acts as a blueprint before coding.

★ 1. Class Diagram

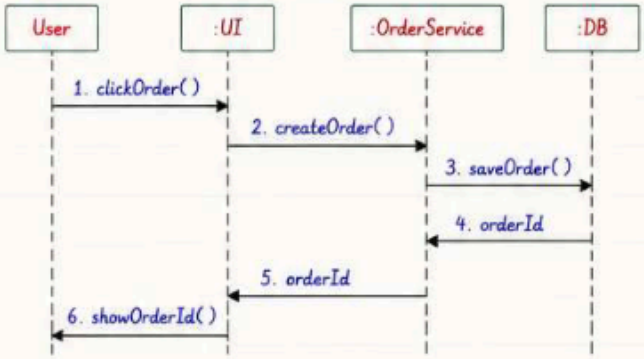
Shows classes, their attributes and methods and relationships between them.



Use :
To show static structure of the system.

★ 2. Sequence Diagram

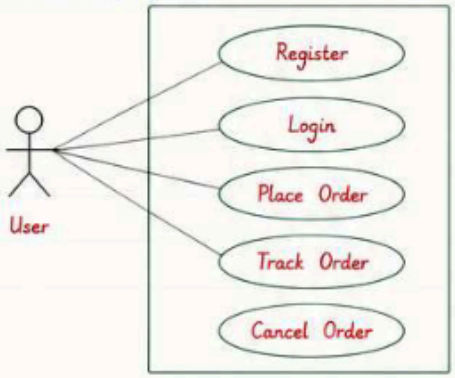
Shows interaction between objects in a particular order (time sequence).



Use :
To show object interactions in a sequence.

★ 3. Use Case Diagram

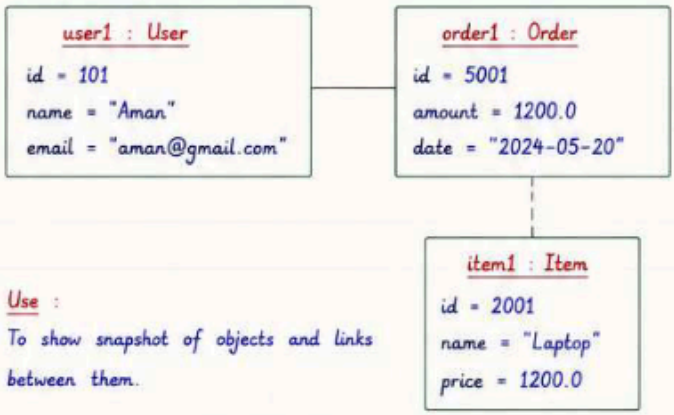
Shows high level functionality of the system from user's perspective.



Use :
To capture requirements and user interactions.

★ 4. Object Diagram

Shows objects and their relationships at a particular point of time.



Use :
To show snapshot of objects and links between them.

★ When to use which UML Diagram ?

Diagram	Use When...
Class Diagram	You want to show structure of classes and relationships.
Sequence Diagram	You want to show how objects interact in a scenario.
Use Case Diagram	You want to capture system features from user's view.
Object Diagram	You want to show real objects and their links at a point of time.

Quick Note :

- ✓ Start with Use Case.
- ✓ Design classes using Class Diagram.
- ✓ Validate flow using Sequence Diagram.
- ✓ Use Object Diagram for real time view.

Low Level Design Handbook

PART 3

3. SOLID Principles (Part 1)

★ What is SOLID ?

SOLID is a set of 5 design principles introduced by Robert C. Martin (Uncle Bob).

These principles help us write maintainable, flexible and scalable code.

SOLID Stands for :

- S - Single Responsibility Principle
- O - Open/Closed Principle
- L - Liskov Substitution Principle
- I - Interface Segregation Principle
- D - Dependency Inversion Principle

★ 1. Single Responsibility Principle (SRP)

A class should have only one reason to change.
It means a class should have only one job / responsibility.

✗ Before (Bad)

User class is handling both user data and email.

```
class User:
  def __init__(self, name, email):
      self.name = name
      self.email = email

  # Saving user to DB
  def save(self):
      pass
  # Sending email
  def send_email(self):
      pass
```

✓ After (Good)

Separate classes for separate responsibilities.

```
class User:
  def __init__(self, name, email):
      self.name = name
      self.email = email
  def save(self):
      pass
```

```
class EmailService:
  def send_email(self, user):
      pass
```

Benefit :

✓ Changes in email logic won't affect user class.

★ 2. Open / Closed Principle (OCP)

Software entities (classes, modules, functions) should be open for extension, but closed for modification.

✗ Before (Bad)

If we add a new shape, we have to modify the AreaCalculatior class.

```
class AreaCalculator:
  def area(self, shape):
      if shape.type == "circle":
      return 3.14 * shape.r * shape.r
      elif shape.type == "rectangle":
      return shape.l * shape.b

  # If new shape added,
  # this class will change again.
```

✓ After (Good)

We extend the system without changing existing code.

```
class Shape:
  def area(self):
      pass
```

```
class Circle(Shape):
  def area(self):
      return 3.14 * self.r * self.r
```

```
class Rectangle(Shape):
  def area(self):
      return self.l * self.b
```

New shapes can be added by extending Shape class.

Benefit :

✓ We can add new functionality without touching existing code.

★ Real World Examples

Principle	Example
SRP	A class should not handle both logging and business logic.
OCP	Payment gateway system where we can add new payment methods easily.

★ Key Takeaways

- ✓ Keep classes small and focused.
- ✓ Separate responsibilities.
- ✓ Extend behavior using inheritance or interfaces.
- ✓ Avoid modifying existing code.
- ✓ These principles lead to clean and robust design.

★ SOLID Benefits

- ✓ Reduces code complexity
- ✓ Improves code readability
- ✓ Easy to test and debug
- ✓ Easy to extend new features
- ✓ Reduces bugs and maintenance cost

★ Quick Comparison

Principle	Focus	Key Idea
Single Responsibility (SRP)	Class	One class → One responsibility
Open/Closed (OCP)	Extensibility	Open for extension, closed for modification

Low Level Design Handbook

PART 4

3. SOLID Principles (Part 2)

★ **Reminder**

SOLID principles help us write clean, maintainable, flexible and scalable code.

SOLID Recap :

S - Single Responsibility Principle
O - Open/Closed Principle
L - Liskov Substitution Principle
I - Interface Segregation Principle
D - Dependency Inversion Principle

★ 3. Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of its subclass without breaking the application.

✗ Before (Bad)

```
class Bird:
    def fly(self):
        pass

class Ostrich(Bird):
    def fly(self):
        raise Exception(
            "I can't fly")
```

✓ After (Good)

```
class Bird:
    def move(self):
        pass

class Sparrow(Bird):
    def move(self):
        print("Flying")

class Ostrich(Bird):
    def move(self):
        print("Running")
```

Benefit :

✓ Subclasses can be used in place of superclass without breaking the system.

★ 4. Interface Segregation Principle (ISP)

No client should be forced to depend on methods it does not use. Create small, specific interfaces.

✗ Before (Bad)

```
class Worker:
    def work(self):
        pass
    def eat(self):
        pass
    def sleep(self):
        pass

class Robot(Worker):
    def work(self):
        pass
    def eat(self):
        pass  # Not needed
    def sleep(self):
        pass  # Not needed
```

✓ After (Good)

```
class Workable:
    def work(self):
        pass

class Eatable:
    def eat(self):
        pass

class Sleepable:
    def sleep(self):
        pass

class Robot(Workable):
    def work(self):
        pass
```

Benefit :

✓ Clients depend only on methods they actually use.

★ 5. Dependency Inversion Principle (DIP)

High level modules should not depend on low level modules. Both should depend on abstractions.

✗ Before (Bad)

```
class MySQLDatabase:
    def connect(self):
        pass

class UserService:
    def __init__(self):
        self.db =
            MySQLDatabase()
```

✓ After (Good)

```
class Database(ABC):
    @abstractmethod
    def connect(self):
        pass

class MySQLDatabase
    (Database):
    def connect(self):
        pass

class UserService:
    def __init__(self,
            db: Database):
        self.db = db
```

Benefit :

✓ Easy to extend, test and maintain. We can switch database without changing UserService.

★ SOLID in One Line

S - One class, one responsibility.
O - Open for extension, closed for modification.
L - Subclasses must be substitutable for base class.
I - Many small interfaces are better than one big interface.
D - Depend on abstractions, not on concrete implementations.

★ How SOLID Helps ?

- Reduces code complexity.
- Improves maintainability.
- Easy to test and debug.
- Easy to extend new features.
- Encourages clean architecture.

Remember :

Follow SOLID + Design Patterns = Clean & Scalable Code ☺

★ Where SOLID is Used ?

- Large scale applications
- Enterprise level systems
- Microservices
- APIs and backend services
- Any project where code must grow and evolve



★ Quick Comparison

Principle	Focus	Key Idea
Liskov Substitution (LSP)	Inheritance	Subclasses should work in place of base class.
Interface Segregation (ISP)	Interfaces	Clients should depend only on required methods.
Dependency Inversion (DIP)	Dependencies	Depend on abstractions, not on concrete classes.

Low Level Design Handbook

PART 5

5. Strategy Pattern

★ What is Strategy Pattern ?

Strategy Pattern defines a family of algorithms, puts each of them in a separate class, and makes them interchangeable.

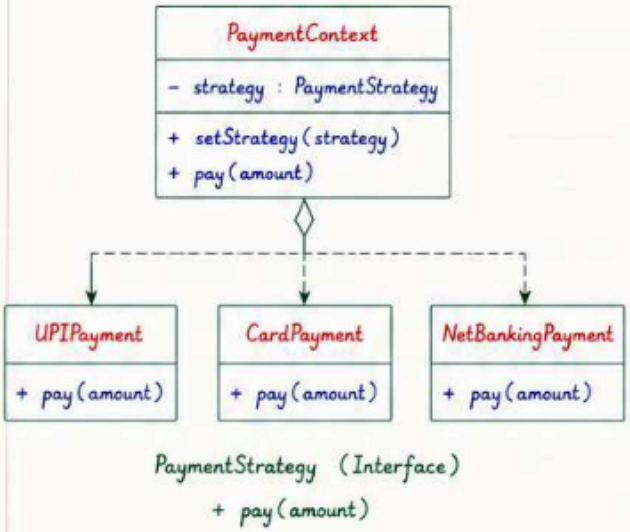
★ When to use ?

- When we have multiple ways to do the same task.
- When we want to switch algorithms at runtime.

★ Example : Payment System

We have different payment methods like UPI, Credit Card, Net Banking. We can switch between them at runtime.

★ UML Class Diagram



★ Key Points

- ✓ Client is independent of concrete strategies.
- ✓ New strategies can be added easily.
- ✓ Follows Open / Closed Principle.

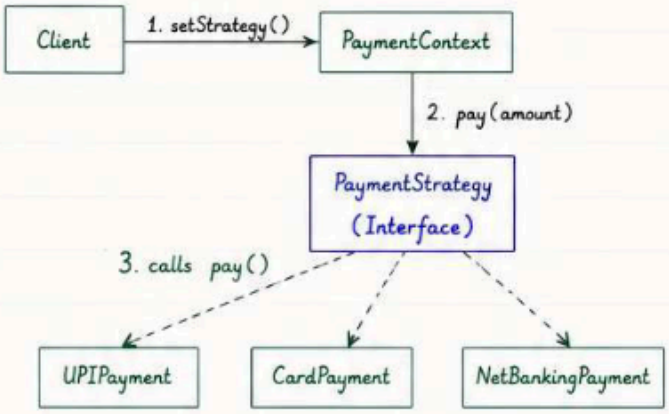
★ How it works ?

① Client creates PaymentContext.

② Client sets a strategy (payment method).

③ Client calls pay(amount).

④ Context calls the pay() method of selected strategy.



★ Code Example (Python)

```
from abc import ABC, abstractmethod

class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class UPIPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid {amount} using UPI")

class CardPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid {amount} using Card")

class PaymentContext:
    def __init__(self, strategy):
        self.strategy = strategy
    def setStrategy(self, strategy):
        self.strategy = strategy
    def pay(self, amount):
        self.strategy.pay(amount)
```

Low Level Design Handbook

PART 6

6. Factory Pattern

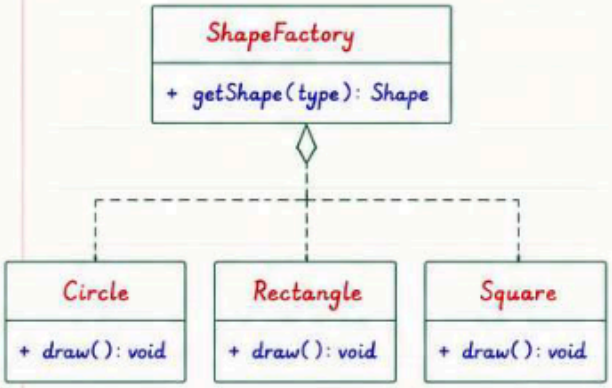
★ What is Factory Pattern ?

Factory Pattern provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

★ When to use ?

- When a class doesn't know which subclass object to create.
- When we want to provide extensibility for creating objects.
- When object creation logic becomes complex.

★ UML Class Diagram



★ Key Points

- ✓ Factory hides the object creation logic.
- ✓ New types can be added without changing client code.
- ✓ Promotes loose coupling.

★ Real World Examples

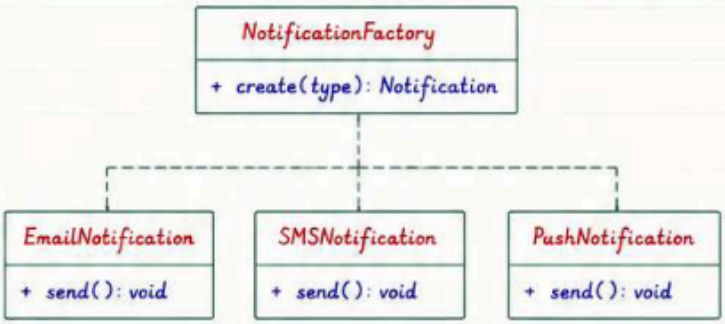
- Notification System
- Logger creation (FileLogger, DBLogger)
- Payment Gateway (UPI, Card, NetBanking)
- Document Export (PDF, Excel, CSV)

★ How it works ?

1. Client asks Factory to create an object.
2. Factory decides which class to instantiate based on input.
3. Factory returns the object to the client.
4. Client uses the object.

★ Example : Notification System

We can create different notifications like Email, SMS, Push using Factory.



★ Code Example (Python)

```
class Notification:
    def send(self):
        pass

class EmailNotification(Notification):
    def send(self):
        print("Sending Email...")

class SMSNotification(Notification):
    def send(self):
        print("Sending SMS...")

class NotificationFactory:
    def create(self, type):
        if type == "email":
            return EmailNotification()
        elif type == "sms":
            return SMSNotification()

factory = NotificationFactory()
n = factory.create("email")
n.send()      # Sending Email...
```

Low Level Design Handbook

PART 7

7. Singleton Pattern

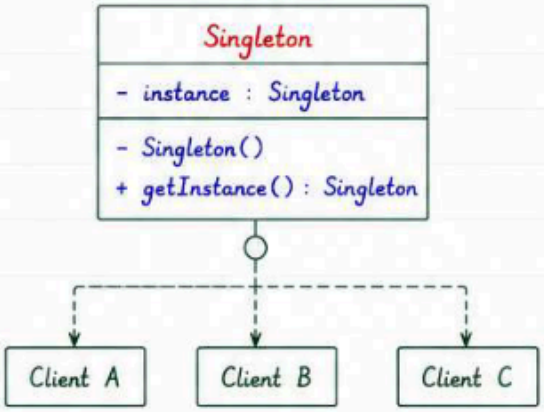
★ What is Singleton Pattern ?

Singleton Pattern ensures that a class has only one instance and provides a global point of access to it.

★ When to use ?

- When exactly one instance of a class is needed.
- When we need a global access point to that instance.

★ UML Class Diagram



★ How it works ?

① Client calls getInstance().
② If instance does not exist, create one.
③ Return the same instance to all clients.

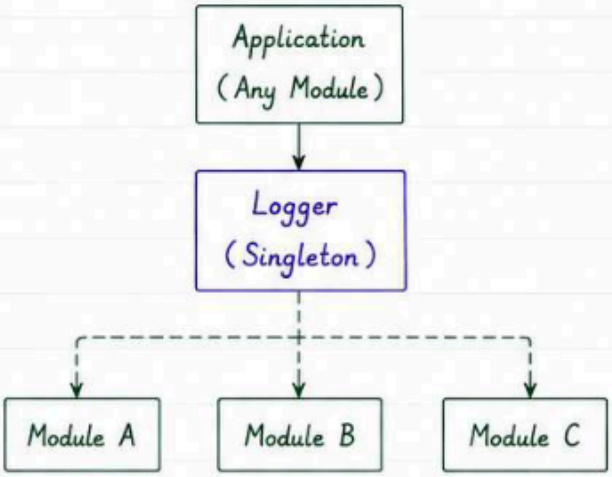
★ Code Example (Python)

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# Usage
obj1 = Singleton()
obj2 = Singleton()
print(obj1 is obj2)    # True
```

★ Example : Logger System

We can use Singleton to create a single logger object that can be used across the application.



★ Benefits

✓ Controlled access to the single instance.
✓ Reduces memory usage.
✓ Provides a global access point.

★ Considerations

- Can make unit testing difficult.
- Can introduce global state (use carefully).

★ Real World Examples

- Logger
- Database Connection
- Configuration Manager
- Cache Manager
- Thread Pool

Low Level Design Handbook

PART 8

8. Observer Pattern

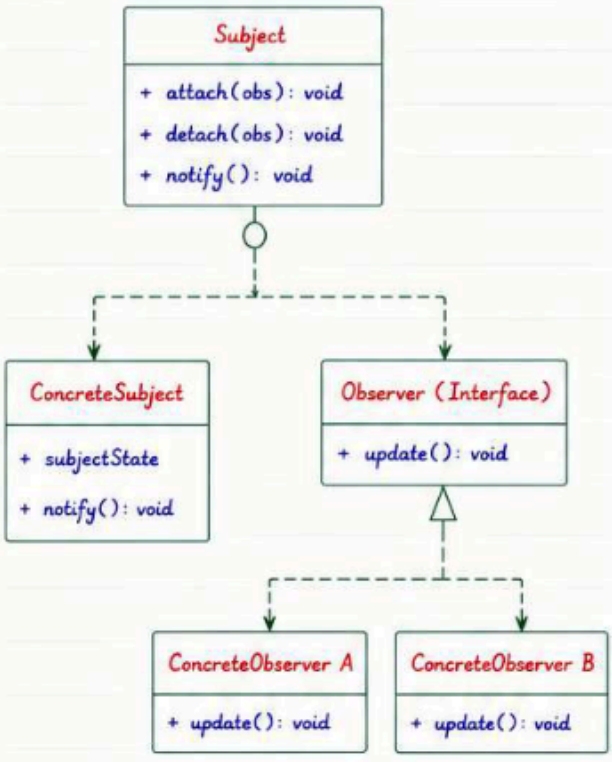
★ What is Observer Pattern ?

Observer Pattern defines a one-to-many dependency between objects. When one object changes state, all its dependents (observers) are notified automatically.

★ When to use ?

- When a change in one object requires changing others.
- When we want to notify multiple objects without tight coupling.
- Common in event handling systems.

★ UML Class Diagram

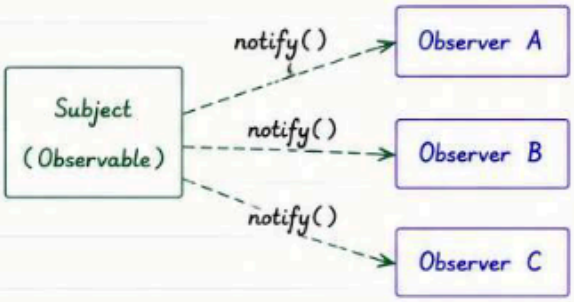


★ Real World Examples

- YouTube channel → Subscribers
- Stock market updates
- Social media notifications
- Event listeners in GUI applications

★ How it works ?

1. Subject (Observable) maintains a list of observers.
2. Observers register with the subject.
3. When subject state changes, it notifies all observers.
4. Observers update themselves.



★ Example : News Channel

A news channel (subject) updates its news.
All subscribers (observers) get the update.

★ Benefits

- Loose coupling between subject and observers.
- Easy to add / remove observers.
- Supports broadcast communication.

★ Code Example (Python)

```
class Subject:
    def __init__(self):
        self._observers = []
    def attach(self, obs):
        self._observers.append(obs)
    def notify(self, message):
        for obs in self._observers:
            obs.update(message)
class Observer:
    def update(self, message):
        pass
class Subscriber(Observer):
    def __init__(self, name):
        self.name = name
    def update(self, message):
        print(f"{self.name} received: {message}")
```

Low Level Design Handbook

PART 9

9. Decorator Pattern

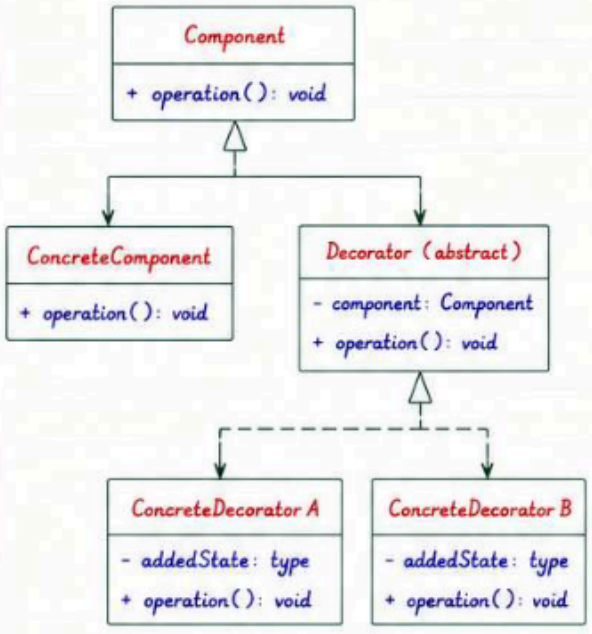
★ What is Decorator Pattern ?

Decorator Pattern allows us to add new functionality to an object dynamically without changing its structure.

★ When to use ?

- When we want to add responsibilities to objects dynamically.
- When inheritance is not a good option (too many subclasses).
- When we need flexible and extensible design.

★ UML Class Diagram

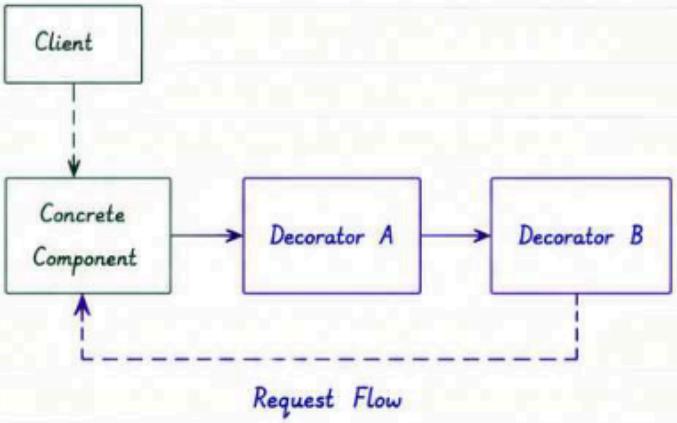


★ Key Points

- ✓ Implements same interface as original object.
- ✓ Wraps the original object.
- ✓ Adds extra behavior before/after calling original object.
- ✓ Can add multiple decorators (stack).

★ How it works ?

1. Client creates a ConcreteComponent object.
2. Client wraps it with one or more decorators.
3. Each decorator adds its own behavior.
4. Request flows through decorators to the original object.



★ Example : Coffee Shop

Base Coffee → Add Milk → Add Sugar → Add Whip

★ Code Example (Java)

```
// Component
interface Coffee {
    String getDescription();
    double getCost();
}

// Concrete Component
class SimpleCoffee implements Coffee {
    public String getDescription() { return "Simple Coffee";}
    public double getCost() { return 50.0; }
}

// Decorator (abstract)
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    public CoffeeDecorator(Coffee coffee) { this.coffee = coffee;}
}
```

(Decorators like Milk, Sugar, Whip will extend CoffeeDecorator and add their own cost & description.)

Low Level Design Handbook

PART 10

10. Adapter Pattern

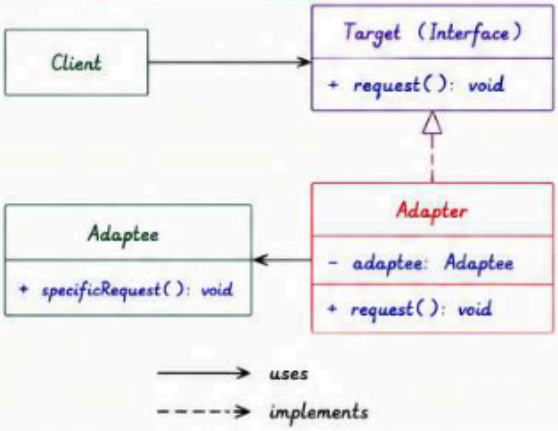
★ What is Adapter Pattern ?

Adapter Pattern allows incompatible interfaces to work together by acting as a bridge between them.

★ When to use ?

- When we want to use an existing class but its interface doesn't match our requirements.
- When we want to reuse legacy code in a new system.
- When two classes have incompatible interfaces.

★ UML Class Diagram



★ Key Points

- ✓ Converts the interface of a class into another interface.
- ✓ Helps in reusing existing code.
- ✓ Promotes code compatibility.
- ✓ Acts as a wrapper.

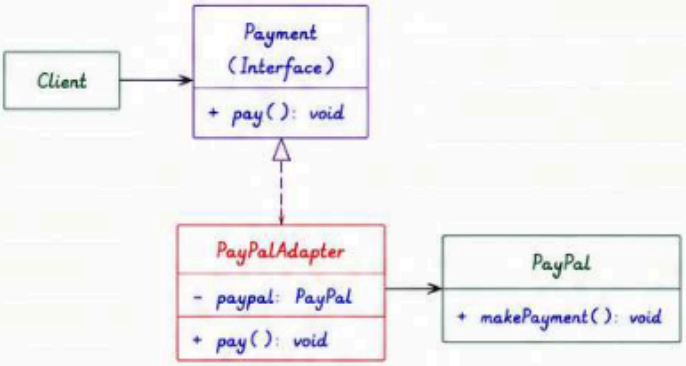
★ Benefits

- Enables classes to work together.
- Improves reusability of existing code.
- Follows Open/Closed Principle.
- Easy to introduce without changing existing code.

★ How it works ?

1. Client calls request() on Target interface.
2. Adapter implements Target and overrides request().
3. Inside request(), Adapter calls the specific method of Adaptee.
4. Adaptee performs the actual work.

★ Example : Payment Gateway



Here PayPal class has makePayment() but our system expects pay(). Adapter bridges this gap.

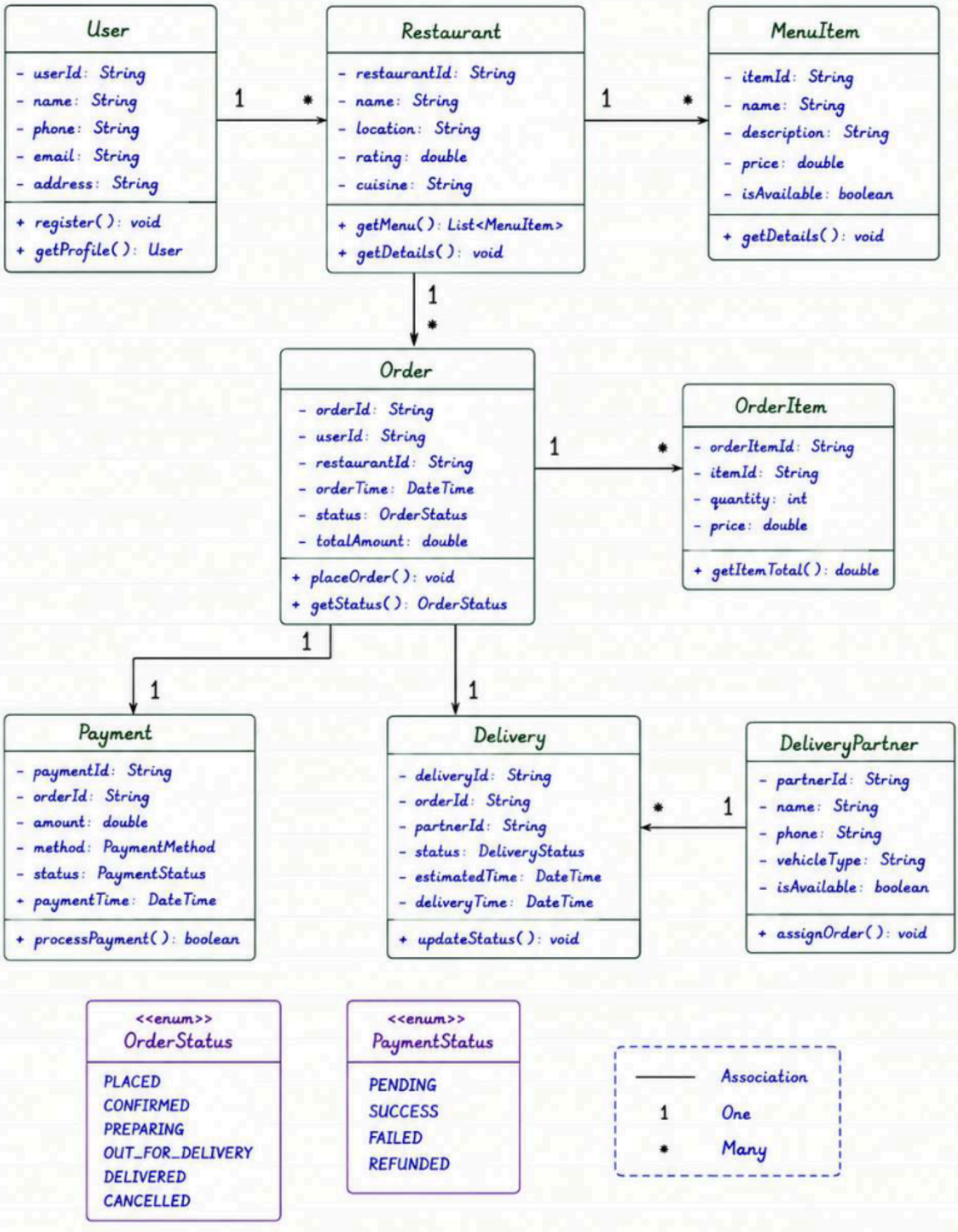
★ Code Example (Java)

```java
// Target Interface
interface Payment {
    void pay(double amount);
}
// Adaptee
class PayPal {
    public void makePayment(double amount) {
        System.out.println("PayPal payment of " + amount);
    }
}
// Adapter
class PayPalAdapter implements Payment {
    private PayPal paypal;

    public PayPalAdapter(PayPal paypal) {
        this.paypal = paypal;
    }
    public void pay(double amount) {
        paypal.makePayment(amount);
    }
}
// Client
class Client {
    public static void main(String[] args) {
        Payment payment = new PayPalAdapter(new PayPal());
        payment.pay(1000.0); // Output: PayPal payment of 1000.0
    }
}
```

Zomato - Low Level Design

Class Diagram (Core Classes)



WhatsApp - Low Level Design

Class Diagram

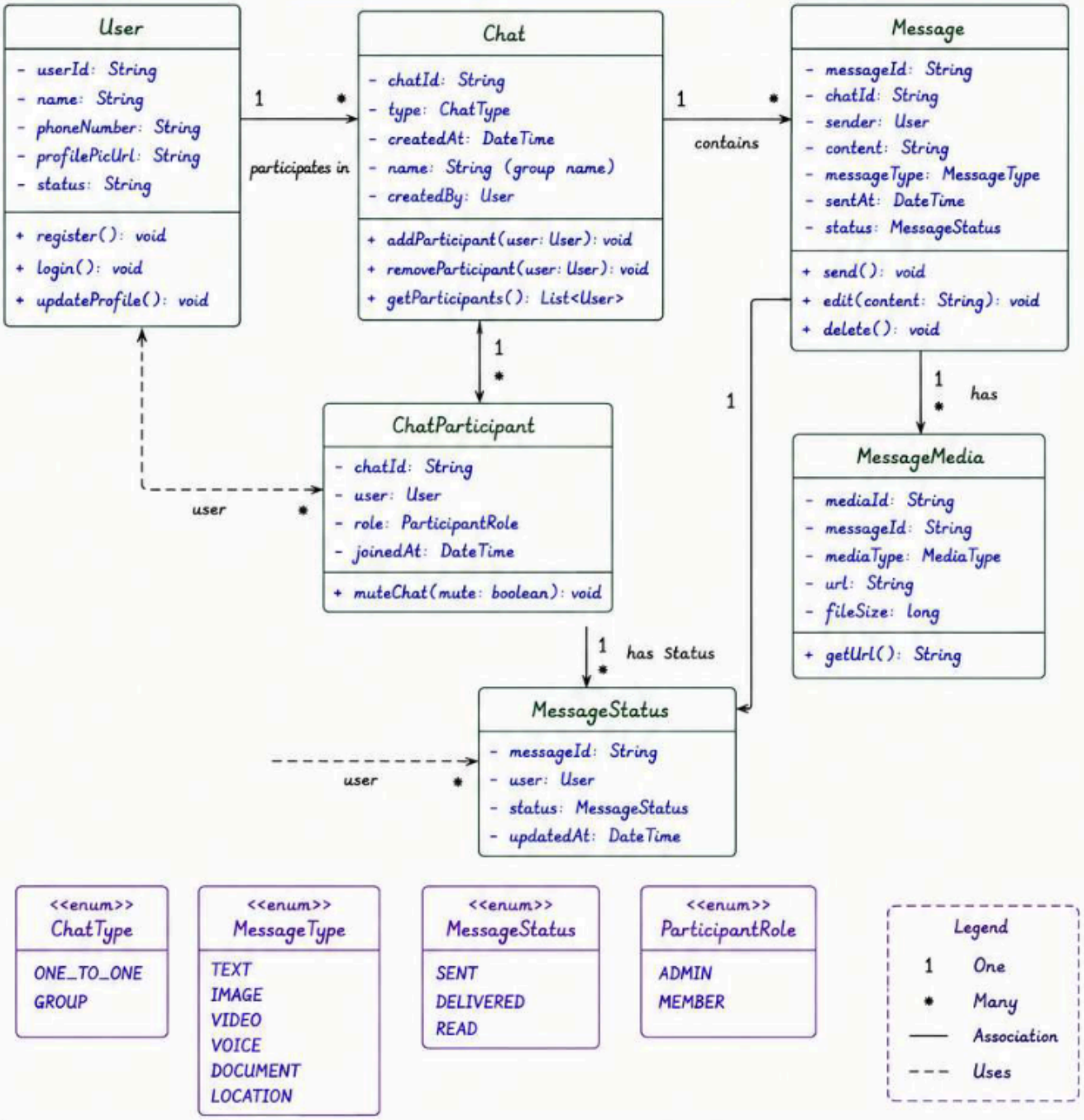
1. Overview

WhatsApp allows users to send text messages, images, videos, voice notes and documents in one-to-one or group chats in real-time. Below diagram shows the core classes and their relationships.

2. Key Features

- User registration & authentication
- One-to-one and group chats
- Real-time messaging
- Message status (sent, delivered, read)
- Media sharing (image, video, file, voice)

3. Class Diagram (Core Classes)



4. Notes

- User can participate in many chats and a chat can have many users.
- Chat of type ONE_TO_ONE will have 2 participants.
- Messages belong to a chat and are sent by a user.
- MessageStatus tracks delivery and read receipts for each user.
- Message may contain media; MessageMedia stores metadata of files.